HAI716I - Projet 1

Louis CABY, Martin GUIONNIERE

31 octobre 2026

1 Architecture

Le robot repose sur quatre rôles : perception, mémoire, décision et action. Ils entourent un monde simulé qui, seul, connaît la vérité (figure 1). À chaque tour, la boucle de `agent.py` enchaîne cinq temps. Le monde renvoie une perception, la mémoire l'intègre, la décision choisit une action, l'actionneur la journalise puis l'exécute, et la décision prend en compte le résultat. Une demande suit une petite machine à états : lecture du dictionnaire, vérification que le résident est atteignable, fouille de l'armoire, livraison.

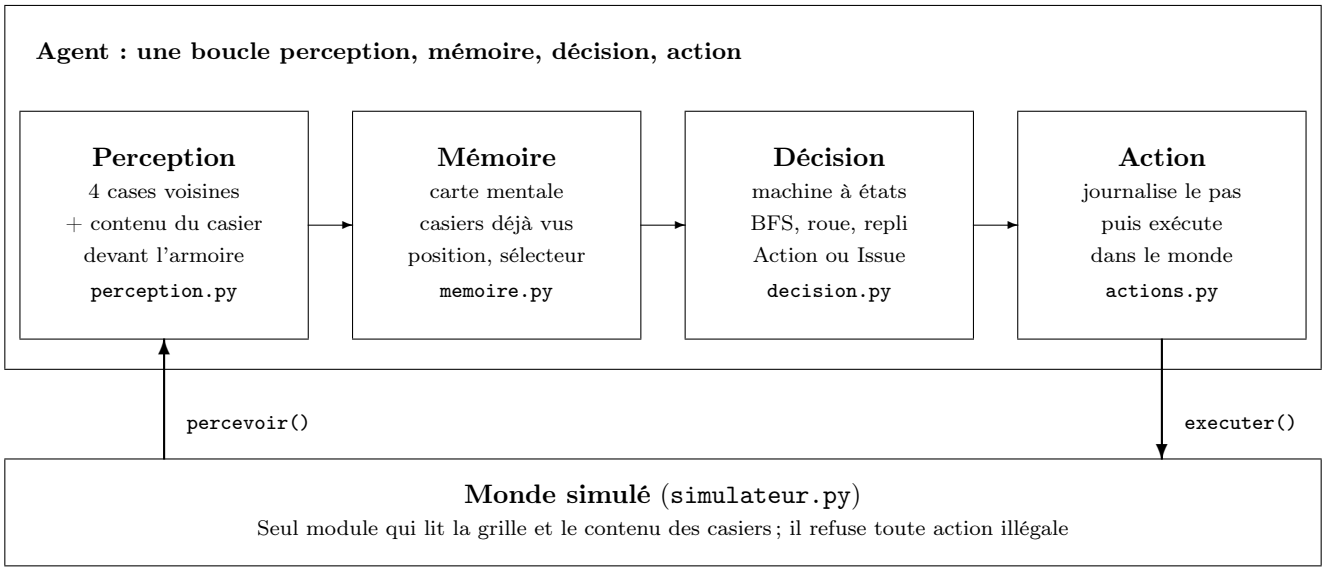


FIGURE 1 – Les quatre rôles du robot et la frontière avec le monde simulé.

Rôle	Modules	Reçoit	Produit	Interdit
Perception	`perception.py`, `Environnement.percevoir`	le monde	les 4 cases voisines, plus le contenu du casier courant devant l'armoire	interpréter, mémoriser
Mémoire	`memoire.py`	perceptions, résultats d'actions	carte mentale, casiers vus, position, sélecteur, objet porté	décider, agir
Décision	`decision.py`, `planification.py`, `roue.py`, `selecteur.py`, `lecture_dictionnaire.py`	la mémoire, en lecture	une `Action` ou une `Issue` (fin de demande)	exécuter, toucher au monde ou aux fichiers
Action	`actions.py`	une `Action`	un `Resultat`, une ligne de trace	choisir

TABLE 1 – Responsabilités et interdits de chaque rôle.

Le robot n'a qu'une source d'information, la `Perception`. Le module `simulateur.py` est le seul à lire `grille` et `objet`. Il ne transmet à l'agent que des `Connaissances` : dimensions, positions des meubles et des résidents, étiquettes de l'armoire, jamais les murs ni le contenu des casiers. Un test lit le code source des modules de l'agent et échoue s'il cite ces champs.

Une perception est vraie mais instantanée. La mémoire est durable mais peut être fausse, car une case inconnue est supposée libre (hypothèse optimiste). Cette séparation permet à la décision de distinguer « vue libre » de « supposée libre ». Elle est indispensable à la preuve qu'un résident est atteignable, qui n'accepte que des cases vues. Pour la même raison, le robot suit lui-même sa position et son sélecteur en comptant ses déplacements, au lieu de les demander au monde.

`decision.py` ne fait aucune entrée-sortie. On teste le plus court chemin, la règle de repli et la lecture du dictionnaire sans simulateur. L'actionneur écrit le pas, avec la position et le casier « avant l'action », puis exécute. La trace décrit donc ce que le robot croyait au moment d'agir. Si l'agent demande une action impossible, le monde lève une `ActionIllegale` : le bug est signalé et non masqué.

La frontière décision/mémoire n'est pas étanche. Le décideur garde une référence vers la mémoire et la lit directement. Il retient aussi lui-même l'émotion lue après `CONSULTER`, au lieu de la confier à la mémoire. La décision ne modifie jamais la mémoire, seulement sa propre fiche de la demande en cours (phase, émotion, casier choisi).

2 Stratégie de re-planification

On utilise une stratégie de re-planification complète basée sur le parcours en largeur. A chaque fois qu'on appelle `aller_adjacent()`, on calcule un nouveau plus court chemin à partir de la position actuelle du robot et des informations connues. Donc quand le robot découvre un nouveau mur qui rends obsolète le trajet initial, il peut recalculer un nouveau trajet sans chercher à réparer son plan actuel. On compare ensuite la longueur du nouveau trajet par rapport à l'ancien pour le signaler dans la trace si elle sont différentes.

La complexité d'un parcours en largeur est en $O(V + E)$ avec V le nombre de sommets (ici les cases explorées) et E le nombre d'arêtes (ici les connections entre les cases). Donc dans le pire des cas on obtient une complexité de $O(N)$, avec N qui est le nombre de cases de la grille. On appelle le parcours en largeur a chaque appel de `aller_adjacent()` donc si on pose K = nombre d'appels, la complexité finale est en $O(K * N)$.

3 Critique de la règle de repli

Elle est déterministe et bornée : elle ne dépasse jamais la distance de 2, donc ne donne jamais l'émotion opposée. Elle est aussi auditable, car l'étiquette `repli` de la trace dit ce qui a été substitué. Enfin elle préfère échouer (« armoire vide ») plutôt que de tendre un objet contraire.

Cependant, elle mesure la ressemblance entre émotions alors qu'il faut choisir un objet adapté à une personne.

- Un objet absurde peut passer avant un objet adapté. Prenons une tristesse dont la colonne est vide dans `armoire_standard`. À distance 1, le sens horaire passe avant l'antihoraire, donc dégoût (colonne 5) passe avant surprise (colonne 3). La personne reçoit un désodorisant, ou des gants de ménage si l'intensité est forte. La colonne surprise est de toute façon entièrement vide dans cette armoire.
- Les meilleurs substituts sont exclus. La confiance (plaid, album souvenir) est à distance 3 de la tristesse, donc interdite. Pourtant un plaid console autant qu'une couverture.
- La symétrie n'est pas justifiée. La distance est symétrique, le besoin ne l'est pas. Un objet de confiance rassure une personne apeurée, l'inverse est moins évident. Le départage horaire/antihoraire n'est qu'un artefact de l'ordre des émotions dans le fichier.
- La substitution est silencieuse. Le résident ne sait pas qu'il reçoit un lot de consolation. Seule la trace le dit.

On peut remplacer la distance sur la roue par une liste ordonnée de substituts par émotion. Elle serait écrite selon la fonction de l'objet (réconforter, rassurer, distraire, apaiser) et non selon la ressemblance des émotions. On conserve le repli d'intensité sur la même émotion, qui reste raisonnable. Si aucun substitut listé n'est disponible, le robot échoue avec un motif explicite, au lieu de tendre un objet inadapté.

Émotion demandée	**Substituts, par ordre**	**Raison**
tristesse	confiance, anticipation	contact et réconfort, puis distraction
peur	confiance	rassurer
colère	confiance	apaiser

Table 2 – Exemple de table de substitution

Cette proposition impliquerait de changer plusieurs fichiers :

— `armoire_*.json` : ajouter un champ `substituts` associant chaque émotion à la liste ordonnée de ses émotions de repli. Le champ serait optionnel : en son absence, on garderait la règle de la roue, ce qui préserve la compatibilité. Sinon il faudrait passer `version` à 2 et modifier `VERSION_ATTENDUE` dans `reconfort_io.py`.

— `validation.py` : vérifier que les clés et les valeurs appartiennent à `emotions`, qu'aucune émotion n'est son propre substitut et qu'une liste ne contient pas de doublon.

— `reconfort_io.py` : ajouter `substitut_1` et `substitut_2` à `REPLIS`, car `voisine_k` décrit la roue.

— `roue.py` : `candidats` lirait la table au lieu de calculer des distances et un tri horaire.

— Vérificateur et tests : réécrire la règle de référence et ajouter des tests d'asymétrie. Tristesse peut recevoir confiance, mais confiance ne doit pas recevoir tristesse.

L'énoncé impose la règle 6.2 pour la notation. Cette proposition reste donc une discussion et n'est pas implémentée dans le rendu.

4 Échecs du robot

À chaque pas on relance une planification complète du trajet, ça ne pose pas trop de problèmes sur des petites cartes mais à partir du moment ou le nombre de cases devient très grand cette stratégie devient un problème. Pour régler ce soucis on pourrait choisir de ne re-planifier un trajet que lorsque celui devient invalide, à cause d'un mur qui apparaît sur le trajet par exemple.

L'itinéraire du sélecteur n'est pas optimal. On cherche à montrer le plus casier non vus ce qui peut nous amener a repasser par des casier vides.

Les trajet ne sont globalement pas optimisés. Il faut vérifier l'accessibilité du résident, aller jusqu'au dictionnaire, aller jusqu'à l'armoire et enfin retourner jusqu'au résident. Pour régler ce problème on pourrait essayer de vérifier l'accessibilité de tous les résidents dès le départ et avoir une mémoire qui nous permettent de retenir la demande de tous les résidents ainsi que le dictionnaire.